

Behaviour-Driven Development (BDD)

Overview

BDD is a software development methodology that bridges the gap between business stakeholders and technical teams. It uses **plain language descriptions** of expected application behavior, often written in **Gherkin syntax**, to define test cases.

Core Concepts

1. **Feature File:** Contains high-level test scenarios written in **Gherkin syntax**.
 - **Given-When-Then** format:
 - **Given:** Pre-condition or context.
 - **When:** Action performed.
 - **Then:** Expected outcome.

Example:

```
Feature: User login
```

```
Scenario: Successful login
  Given the user is on the login page
  When the user enters valid credentials
  Then the user should be redirected to the dashboard
```

2. **Step Definitions:** Maps each Gherkin step to a function that performs the action.
 3. **Test Runner:** Executes the feature file by invoking corresponding step definitions.
-

Example Implementation in Python

Using **Behave** framework:

Feature File

```
Feature: User login functionality
```

```
Scenario: Successful login
  Given the user is on the login page
  When the user enters valid credentials
  Then the user should be redirected to the dashboard
```

Step Definitions

```
from behave import given, when, then
from selenium import webdriver

driver = None
```

```

@given("the user is on the login page")
def step_given_user_on_login_page(context):
    global driver
    driver = webdriver.Chrome()
    driver.get("https://example.com/login")

@when("the user enters valid credentials")
def step_when_user_enters_credentials(context):
    driver.find_element_by_id("username").send_keys("test_user")
    driver.find_element_by_id("password").send_keys("secure_password")
    driver.find_element_by_id("login-button").click()

@then("the user should be redirected to the dashboard")
def step_then_user_redirected_to_dashboard(context):
    assert "dashboard" in driver.current_url
    driver.quit()

```

Running Tests

Run the tests using the `behave` command:

```
behave
```

Example Implementation in Java

Using **Cucumber** framework:

Feature File

```

Feature: User login functionality

    Scenario: Successful login
        Given the user is on the login page
        When the user enters valid credentials
        Then the user should be redirected to the dashboard

```

Step Definitions

```

import io.cucumber.java.en.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginSteps {
    WebDriver driver;

    @Given("the user is on the login page")
    public void the_user_is_on_the_login_page() {
        driver = new ChromeDriver();
        driver.get("https://example.com/login");
    }

    @When("the user enters valid credentials")
    public void the_user_enters_valid_credentials() {
        driver.findElement(By.id("username")).sendKeys("test_user");
    }
}

```

```

        driver.findElement(By.id("password")).sendKeys("secure_password");
        driver.findElement(By.id("login-button")).click();
    }

    @Then("the user should be redirected to the dashboard")
    public void the_user_should_be_redirected_to_the_dashboard() {
        String currentUrl = driver.getCurrentUrl();
        assert currentUrl.contains("dashboard");
        driver.quit();
    }
}

```

Test Runner

```

import org.junit.runner.RunWith;
import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(features = "src/test/resources/features", glue =
    "stepDefinitions")
public class TestRunner {
}

```

Running Tests

Run tests with Maven:

```
mvn test
```

Example Implementation in JavaScript

Using Cypress with Cucumber Preprocessor:

Feature File

Feature: User login functionality

```

Scenario: Successful login
    Given the user is on the login page
    When the user enters valid credentials
    Then the user should be redirected to the dashboard

```

Step Definitions

```

import { Given, When, Then } from "@badeball/cypress-cucumber-preprocessor";

Given("the user is on the login page", () => {
    cy.visit("https://example.com/login");
});

When("the user enters valid credentials", () => {
    cy.get("#username").type("test_user");
    cy.get("#password").type("secure_password");
    cy.get("#login-button").click();
}

```

```
});  
  
Then("the user should be redirected to the dashboard", () => {  
    cy.url().should("include", "dashboard");  
});
```

Cypress Configuration

Install the necessary dependencies:

```
npm install @badeball/cypress-cucumber-preprocessor
```

Update `cypress.config.js`:

```
const cucumber = require("@badeball/cypress-cucumber-preprocessor").default;  
  
module.exports = {  
    e2e: {  
        setupNodeEvents(on, config) {  
            on("file:preprocessor", cucumber());  
        },  
        specPattern: "cypress/e2e/**/*.feature",  
    },  
};
```

Running Tests

Run Cypress with:

```
npx cypress open
```

Advantages of BDD

1. **Collaboration:** Promotes understanding between business and technical teams.
2. **Readability:** Non-technical stakeholders can understand the scenarios.
3. **Reusability:** Step definitions can be reused across multiple scenarios.